

TP Guide : Unity & IA

🔗 Pré-requis Unity

Ce guide suppose que vous avez une scène Unity avec :

1. Un Circuit fermé (avec des murs/barrières et des Colliders).
2. Un Prefab de Voiture fonctionnel (basé sur le TP Session 18 ou WheelCollider).
3. Un script de contrôle qui accepte acceleration et steering en paramètres publics.

1. Le Cerveau (Script NeuralNetwork.cs)

Ne perdez pas de temps à coder la multiplication matricielle. Voici une classe C# simplifiée prête à l'emploi. Elle gère un réseau à une couche cachée.

NeuralNetwork.cs

```
[System.Serializable]
public class NeuralNetwork {
    public int[] layers; // Ex: [5, 8, 2] (5 entrées, 8 cachés, 2 sorties)
    public float[][] neurons;
    public float[][][] weights;

    public NeuralNetwork(int[] layers) {
        this.layers = new int[layers.Length];
        for (int i = 0; i < layers.Length; i++) this.layers[i] = layers[i];
        InitNeurons();
        InitWeights();
    }

    // Propagation avant (Feed Forward)
    public float[] FeedForward(float[] inputs) {
        for (int i = 0; i < inputs.Length; i++) neurons[0][i] = inputs[i];

        for (int i = 1; i < layers.Length; i++) {
            for (int j = 0; j < neurons[i].Length; j++) {
                float value = 0f;
                for (int k = 0; k < neurons[i-1].Length; k++)
                    value += weights[i-1][j][k] * neurons[i-1][k];
                neurons[i][j] = (float)Math.Tanh(value); // Activation (-1 à 1)
            }
        }
        return neurons[neurons.Length - 1]; // Retourne la couche de sortie
    }

    // Mutation (Pour l'algo génétique)
    public void Mutate(float probability, float amount) {
        for (int i = 0; i < weights.Length; i++) {
            for (int j = 0; j < weights[i].Length; j++) {
                for (int k = 0; k < weights[i][j].Length; k++) {
                    if (UnityEngine.Random.value < probability)
                        weights[i][j][k] += UnityEngine.Random.Range(-amount,
amount);
                }
            }
        }
    }
}
```

```

    }

    // (Ajoutez InitNeurons et InitWeights ici - Initialisation aléatoire simple)
}

```

2. Les Yeux (Raycasts)

Dans votre script CarAI.cs, vous devez collecter les données de l'environnement.

Logique des Capteurs. Lancez 5 rayons. Pour chaque rayon, normalisez la distance :

- Si impact à 0m -> Input = 1 (Danger immédiat).
- Si impact à MaxDist -> Input = 0 (Voie libre).
- Si pas d'impact -> Input = 0.

```

float[] inputs = new float[5];
float maxDist = 20f;
Vector3[] directions = { -transform.right, (-transform.right +
transform.forward).normalized, transform.forward, (transform.right +
transform.forward).normalized, transform.right };

for (int i = 0; i < 5; i++) {
    inputs[i] = 0; // Par défaut : rien
    if (Physics.Raycast(transform.position, directions[i], out RaycastHit hit,
maxDist)) {
        inputs[i] = 1f - (hit.distance / maxDist); // Inversion : 1 = proche
        Debug.DrawLine(transform.position, hit.point, Color.red);
    }
}

```

3. Le Contrôleur (Brain -> Moteur)

Connectez le cerveau à la voiture dans FixedUpdate.

```

// 1. Demander au cerveau
float[] outputs = brain.FeedForward(inputs);

// 2. Interpréter les sorties (Tanh renvoie entre -1 et 1)
float acceleration = outputs[0]; // > 0 avancer, < 0 reculer
float steering = outputs[1];     // > 0 droite, < 0 gauche

// 3. Envoyer au script physique (Session 18)
carController.Move(acceleration, steering);

```

4. Le Manager Génétique (GeneticManager.cs)

C'est le chef d'orchestre. Il ne doit y en avoir qu'un seul dans la scène.

4.1 Initialisation

Au Start, instanciez 20 voitures (Instantiate). Stockez-les dans une liste. Donnez-leur un cerveau aléatoire new NeuralNetwork(new int[] {5, 8, 2}).

4.2 Gestion de la mort

Dans le script de la voiture, détectez la collision (OnCollisionEnter).

- Si collision avec "Mur" : hasCrashed = true. Désactivez la voiture.
- Le Manager doit vérifier à chaque frame : "Reste-t-il des voitures vivantes ?"

4.3 Nouvelle Génération (Repopulate)

Quand tout le monde est mort :

1. Trier : Trouvez la voiture qui a le meilleur fitness (Distance parcourue + Vitesse moyenne).
2. Sauvegarder : Copiez son cerveau (Deep Copy des poids !).
3. Cloner : Réinitialisez les 20 voitures au point de départ.
4. Appliquer :
 - Voiture 0 : Cerveau du champion (sans mutation) -> Pour ne pas régresser.
 - Voitures 1 à 19 : Cerveau du champion + `Mutate(0.05f, 0.5f)`.

Astuce : Accélérer le temps

Pour entraîner l'IA plus vite, ajoutez un Slider dans l'UI qui modifie `Time.timeScale`. Vous pouvez monter jusqu'à `Time.timeScale = 10` ou `20` pour voir les générations défiler à toute vitesse !